

Module-4 Assignment DBMS

Theory Question..

1. What is SQL, and why is it essential in database management?

SQL (Structured Query Language) is a standardized programming language used to create, manage, and manipulate databases. It allows users to store, retrieve, update, and delete data efficiently.

Why SQL is essential:

It provides a consistent way to interact with databases.

It supports complex queries to extract meaningful information.

It enables data security through permissions and roles.

It helps maintain data integrity with constraints (e.g., PRIMARY KEY, FOREIGN KEY).

It is used by almost all modern relational database systems (MySQL, PostgreSQL, Oracle, SQL Server).

2. Explain the difference between DBMS and RDBMS.

DBMS	RDBMS
Stands for Database Management System	Stands for Relational Database Management System
Stores data as files or hierarchical structures	Stores data in tables (rows and columns)
No relationships between data	Relationships maintained using keys (primary & foreign keys)
Does not support normalization fully	Supports normalization to reduce redundancy
Examples: File systems, XML databases	Examples: MySQL, PostgreSQL, Oracle, SQL Server

In short:

DBMS manages databases in general, while RDBMS is a type of DBMS specifically designed to store and manage data in a structured, relational (table-based) format.

3. Describe the role of SQL in managing relational databases.

SQL is the primary language used to interact with RDBMS. It performs tasks such as:

Data Definition (DDL):

Create, alter, and delete database structures (e.g., CREATE TABLE, ALTER TABLE).

Data Manipulation (DML):

Insert, update, delete, and retrieve data (e.g., INSERT, UPDATE, DELETE, SELECT).

Data Control (DCL):

Control access and permissions (e.g., GRANT, REVOKE).

Transaction Control (TCL):

Manage transactions and ensure data consistency (e.g., COMMIT, ROLLBACK).

Summary: SQL acts as the bridge between the user/application and the database, enabling efficient data management and retrieval.

4. What are the key features of SQL?

Key features include:

Simple and Easy to Learn – Uses English-like syntax.

Data Querying – Powerful SELECT statements for complex queries.

Data Definition – Ability to define and modify database structures.

Data Manipulation – Insert, update, delete, and manage data.

Data Control and Security – Role-based permissions and access control.

Transaction Control – Ensures data accuracy through COMMIT and ROLLBACK.

Portability – Works across different RDBMS platforms with minimal changes.

Support for Joins and Relationships – Allows combining multiple tables using keys.

Scalability – Efficient for large datasets and enterprise-level applications.

1. What are the basic components of SQL syntax?

The basic components of SQL syntax include:

1. **Keywords** – Reserved words that perform specific operations (e.g., `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `WHERE`, `FROM`).
2. **Identifiers** – Names of database objects (tables, columns, views).
3. **Expressions** – Combinations of values, operators, and functions that evaluate to a value.
4. **Clauses** – Components of SQL statements that specify conditions or modify actions (e.g., `WHERE`, `GROUP BY`).
5. **Operators** – Symbols used for comparisons or calculations (`=`, `<`, `>`, `AND`, `OR`, `LIKE`).
6. **Literals** – Constant values such as numbers or strings (`100`, `'John'`).
7. **Functions** – Built-in routines to manipulate data (`COUNT()`, `SUM()`, `NOW()`).

2. Write the general structure of an SQL SELECT statement.

The basic structure is:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column  
HAVING group_condition  
ORDER BY column ASC|DESC;
```

Notes:

- Only **SELECT** and **FROM** are mandatory.
- Other clauses are optional and used as needed.

3. Explain the role of clauses in SQL statements.

Clauses are building blocks of SQL statements that **define specific parts of the query**.

They help control how data is selected, filtered, grouped, or sorted.

Roles of common clauses:

- **FROM** → Specifies the table(s) to retrieve data from.
- **WHERE** → Filters rows based on a condition.

- **GROUP BY** → Groups rows that have the same values in specified columns.
- **HAVING** → Filters groups created by **GROUP BY**.
- **ORDER BY** → Sorts the final result set.
- **LIMIT / OFFSET** → Restricts the number of rows returned.

In summary:

Clauses allow SQL queries to be more precise, structured, and powerful in retrieving or manipulating data.

1. What are constraints in SQL? List and explain the different types of constraints.

Constraints in SQL are rules applied to table columns to ensure data accuracy, validity, and integrity. They restrict the type of data that can be inserted into a table.

Types of SQL Constraints:

PRIMARY KEY

Uniquely identifies each row in a table.

Combines **UNIQUE** + **NOT NULL**.

FOREIGN KEY

Establishes a relationship between two tables.

Ensures that values in one table match values in another table's primary key.

UNIQUE

Ensures all values in a column are different.

NOT NULL

Ensures a column cannot have empty (NULL) values.

CHECK

Ensures values meet a specified condition.

Example: age >= 18.

DEFAULT

Assigns a default value to a column if no value is provided.

INDEX (not technically a constraint, but often listed)

Improves search performance on columns.

2. How do PRIMARY KEY and FOREIGN KEY constraints differ?

PRIMARY KEY FOREIGN KEY

Uniquely identifies each record in a table Creates a link between two tables

Cannot contain NULL values Can contain NULL (unless restricted)

Only one primary key allowed per table Multiple foreign keys can exist

Defined on the parent table Defined in the child table

Ensures entity integrity Ensures referential integrity

Summary:

A primary key uniquely identifies a record, while a foreign key connects tables and ensures valid relationships.

3. What is the role of NOT NULL and UNIQUE constraints?

NOT NULL Constraint

Ensures a column must always have a value.

Prevents inserting or updating rows with NULL values.

Used when a field is mandatory (e.g., name, price).

UNIQUE Constraint

Ensures no two values in a column are the same.

Allows NULL values (except when combined with NOT NULL).

Used for fields that must be distinct (e.g., email, username).

In short:

NOT NULL → Prevents empty values.

UNIQUE → Prevents duplicate values.

1. Define the SQL Data Definition Language (DDL).

DDL (Data Definition Language) in SQL consists of commands used to define, create, modify, and delete the structure of database objects such as tables, indexes, and views.

Common DDL commands include:

CREATE – create database objects

ALTER – modify existing structures

DROP – delete objects

TRUNCATE – remove all rows from a table

RENAME – rename database objects

In short:

DDL is responsible for managing database structure, not the data inside it.

2. Explain the CREATE command and its syntax.

The CREATE command is used to create new database objects such as tables, databases, views, and indexes.

The most common use is creating a table.

Syntax of CREATE TABLE:

```
CREATE TABLE table_name (  
    column1 data_type constraint,  
    column2 data_type constraint,  
    column3 data_type constraint,  
    ...  
);
```

Example:

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Email VARCHAR(100) UNIQUE  
);
```

This command creates a table with columns, data types, and constraints.

3. What is the purpose of specifying data types and constraints during table creation?

Purpose of Data Types:

Define what kind of data a column can store (e.g., INT, VARCHAR).

Ensure storage efficiency.

Prevent invalid data from being entered.

Improve performance and query accuracy.

Purpose of Constraints:

Maintain data integrity (correctness and validity).

Prevent incorrect, duplicate, or inconsistent data.

Enforce rules such as uniqueness, mandatory values, and valid relationships.

Support reliable database operations.

Summary:

Data types ensure correct format of values.

Constraints ensure correct rules are followed.

1. What is the use of the ALTER command in SQL?

The ALTER command in SQL is used to modify the structure of an existing table without deleting or recreating it.

It allows you to:

Add new columns

Modify existing columns (data type, size, constraints)

Rename columns or tables

Drop (delete) columns

Add or remove constraints

In short:

ALTER helps you update the table structure after it has been created.

2. How can you add, modify, and drop columns from a table using ALTER?

a) Add a Column

```
ALTER TABLE table_name  
ADD column_name data_type constraint;
```

Example:

```
ALTER TABLE Employees  
ADD Salary INT;
```

b) Modify (Change) a Column

Different SQL systems use slightly different syntax.
Common syntax:

```
ALTER TABLE table_name  
MODIFY column_name new_data_type constraint;
```

Example:

```
ALTER TABLE Employees  
MODIFY Salary DECIMAL(10,2);
```

In SQL Server:

```
ALTER TABLE Employees  
ALTER COLUMN Salary DECIMAL(10,2);
```

c) Drop (Delete) a Column

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

Example:

ALTER TABLE Employees

DROP COLUMN Salary;

Summary Table

Operation SQL Command

Add column ALTER TABLE ... ADD

Modify column ALTER TABLE ... MODIFY or ALTER COLUMN

Drop column ALTER TABLE ... DROP COLUMN

1. Define the INSERT, UPDATE, and DELETE commands in SQL.

INSERT Command

The INSERT command is used to add new records (rows) into a table.

Example:

```
INSERT INTO Students (ID, Name, Age)
```

```
VALUES (1, 'John', 20);
```

UPDATE Command

The UPDATE command is used to modify existing records in a table.

Example:

```
UPDATE Students
```

```
SET Age = 21
```

```
WHERE ID = 1;
```

DELETE Command

The DELETE command is used to remove existing records from a table.

Example:

```
DELETE FROM Students
```

```
WHERE ID = 1;
```

Summary:

INSERT → adds data

UPDATE → modifies data

DELETE → removes data

2. What is the importance of the WHERE clause in UPDATE and DELETE operations?

The WHERE clause is used to specify which rows should be updated or deleted.

Importance:

Prevents unwanted changes

Without WHERE, the command applies to all rows in the table.

Ensures data accuracy

Only the intended records are affected.

Protects against data loss

Especially in DELETE operations—omitting WHERE can delete the entire table's data.

Examples:

Without WHERE (dangerous):

```
UPDATE Employees SET Salary = 50000;
```

→ Updates salary of all employees.

With WHERE (safe):

```
DELETE FROM Employees WHERE EmpID = 10;
```

→ Deletes only one specific record.

In short:

WHERE ensures that UPDATE and DELETE apply to only the rows you want and protects the database from accidental changes or data loss.

1. What is the SELECT statement, and how is it used to query data?

The SELECT statement is the most commonly used SQL command. It is used to retrieve data from one or more tables in a database.

Purpose:

To extract specific information from the database.

To display selected columns or filtered rows.

To perform calculations, sorting, grouping, and joins.

Basic Syntax:

```
SELECT column1, column2, ...  
FROM table_name;
```

Example:

```
SELECT Name, Age  
FROM Students;
```

This retrieves the Name and Age columns from the Students table.

In simple words:

SELECT is used to query and display data stored in tables.

2. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

a) WHERE Clause

The WHERE clause is used to filter rows based on a given condition.

Purpose of WHERE:

Retrieve only the records that meet a specific condition.

Limit data shown in a query.

Used commonly with SELECT, UPDATE, and DELETE.

Example:

```
SELECT * FROM Employees  
WHERE Salary > 30000;
```

This retrieves only employees whose salary is greater than 30,000.

b) ORDER BY Clause

The ORDER BY clause is used to sort the result set of a query.

Purpose of ORDER BY:

Arrange data in ascending (ASC) or descending (DESC) order.

Helps present data more clearly.

Example:

```
SELECT Name, Salary  
FROM Employees  
ORDER BY Salary DESC;
```

This displays employees sorted by salary in descending order.

In summary:

WHERE → filters rows

ORDER BY → sorts rows

Together, they make SQL queries more powerful and flexible.

1. What is the purpose of GRANT and REVOKE in SQL?

GRANT

The GRANT command is used to give permissions (privileges) to users on database objects such as tables, views, or procedures.

It allows users to perform specific operations like SELECT, INSERT, UPDATE, or DELETE.

REVOKE

The REVOKE command is used to remove permissions that were previously granted to users.

It restricts users from performing certain database operations.

In short:

GRANT → gives access

REVOKE → removes access

2. How do you manage privileges using these commands?

You can manage user privileges by assigning or removing access rights using GRANT and REVOKE.

a) Granting Privileges

Syntax:

GRANT privilege ON object_name TO user_name;

Example:

Give SELECT and INSERT permissions on the Students table:

GRANT SELECT, INSERT ON Students TO user1;

b) Revoking Privileges

Syntax:

REVOKE privilege ON object_name FROM user_name;

Example:

Remove INSERT permission from user1:

REVOKE INSERT ON Students FROM user1;

Privileges You Can Grant/Revoke Include:

SELECT

INSERT

UPDATE

DELETE

ALTER

ALL PRIVILEGES (gives all permissions)

In summary:

Use GRANT to give users access to database operations.

Use REVOKE to take away those permissions when needed.

1. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

COMMIT

The COMMIT command is used to save all changes made during the current transaction permanently in the database.

Once committed, the changes cannot be undone.

Example:

COMMIT;

ROLLBACK

The ROLLBACK command is used to undo changes made during the current transaction.

It restores the database to the state before the transaction began.

Example:

ROLLBACK;

In short:

COMMIT → saves changes permanently

ROLLBACK → cancels/undoes changes

2. Explain how transactions are managed in SQL databases.

A transaction is a sequence of one or more SQL operations that are treated as a single logical unit of work.

Key points on transaction management:

a) Transactions use ACID properties:

Atomicity – All operations succeed or all fail together.

Consistency – Database remains in a valid state before and after the transaction.

Isolation – Transactions do not interfere with each other.

Durability – Once committed, changes persist even after system failures.

b) How Transactions Work:

Start of Transaction

Happens automatically when a DML operation (INSERT, UPDATE, DELETE) begins.

Or manually using:

START TRANSACTION;

Perform SQL Operations

Example:

UPDATE Accounts SET Balance = Balance - 100 WHERE ID = 1;

Commit the Transaction

Saves all changes permanently:

COMMIT;

Rollback if Needed

If an error occurs or conditions fail:

ROLLBACK;

c) Savepoints (Optional Feature)

SAVEPOINT allows rolling back part of a transaction.

Example:

SAVEPOINT sp1;

ROLLBACK TO sp1;

1. Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

What is a JOIN?

A JOIN in SQL is used to combine rows from two or more tables based on a related column between them (usually a primary key in one table and a foreign key in another).

Joins allow you to retrieve connected data stored in separate tables.

Types of Joins and Their Differences

1. INNER JOIN

Returns only the matching rows from both tables.

Rows without a match are excluded.

Example output:

✓ Matches only

✗ No unmatched rows

2. LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table and matching rows from the right table.

If no match exists, NULL values appear for the right table's columns.

Example output:

- ✓ All left table rows
- ✓ Matching right rows
- ✗ Non-matching right rows

3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table.

If no match exists, NULL appears for the left table's columns.

Example output:

- ✓ All right table rows
- ✓ Matching left rows
- ✗ Non-matching left rows

4. FULL OUTER JOIN

Returns all rows from both tables.

Matching rows are combined.

Non-matching rows appear with NULLs.

Example output:

- ✓ All left rows
- ✓ All right rows
- ✓ Matching and non-matching combinations

Summary Table of Joins

Join Type	Returns
-----------	---------

INNER JOIN	Only matching rows
------------	--------------------

LEFT JOIN	All left rows + matching right rows
-----------	-------------------------------------

RIGHT JOIN	All right rows + matching left rows
------------	-------------------------------------

FULL OUTER JOIN	All rows from both tables
-----------------	---------------------------

2. How are joins used to combine data from multiple tables?

Joins use a common column (usually keys) to fetch related data from multiple tables.

How it works:

SQL compares values in the related columns of the two tables.

Based on the join type, it selects rows that match or do not match.

The result is a single combined table.

Example:

Suppose you have two tables:

Students Table

StudentID	Name
-----------	------

Marks Table

StudentID	Marks
-----------	-------

Joining them:

```
SELECT Students.Name, Marks.Marks
FROM Students
INNER JOIN Marks
ON Students.StudentID = Marks.StudentID;
```

1. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

GROUP BY Clause

- The **GROUP BY** clause in SQL is used to **group rows that have the same values in one or more columns**.
- It is often used with **aggregate functions** to perform calculations on each group of rows rather than on the entire table.

Common Aggregate Functions

- **COUNT()** → Counts the number of rows
- **SUM()** → Calculates the total sum
- **AVG()** → Calculates the average
- **MAX()** → Finds the maximum value
- **MIN()** → Finds the minimum value

Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1;
```

Example

Suppose we have a **Sales** table:

Product	Quantity
Pen	10
Pencil	20

Pen 15

Pencil 10

Query to get total quantity per product:

```
SELECT Product, SUM(Quantity) AS TotalQuantity
FROM Sales
GROUP BY Product;
```

Result:

Product	TotalQuantity
Pen	25
Pencil	30

Explanation:

- Rows with the same product are **grouped together**, and the `SUM()` function calculates the total quantity per product.

2. Explain the difference between GROUP BY and ORDER BY

Feature	GROUP BY	ORDER BY
Purpose	Groups rows based on column values	Sorts rows in ascending or descending order
Used With	Often used with aggregate functions (SUM, COUNT, AVG)	Can be used with or without aggregate functions

Result	Returns one row per group	Returns all rows but sorted
Execution	Groups data before performing calculations	Orders the final result set

Example Combining Both

```
SELECT Product, SUM(Quantity) AS TotalQuantity
FROM Sales
GROUP BY Product
ORDER BY TotalQuantity DESC;
```

- First, rows are **grouped by Product**.
- Then, the grouped results are **sorted** by TotalQuantity in descending order.

In summary:

- **GROUP BY** → groups rows for aggregation.
- **ORDER BY** → sorts rows for presentation.

1. What is a stored procedure in SQL, and how does it differ from a standard SQL query?

Stored Procedure

A **stored procedure** is a **precompiled set of SQL statements** stored in the database that can be **executed as a single unit**. It can accept parameters, perform operations, and return results.

Key Differences from Standard SQL Query:

Feature	Stored Procedure	Standard SQL Query
Definition	Predefined set of SQL statements stored in the database	Ad-hoc SQL commands executed one at a time
Reusability	Can be called multiple times from applications or scripts	Written and executed each time separately
Parameters	Can accept input and output parameters	Usually does not accept parameters unless embedded in application logic
Performance	Precompiled, faster for repeated execution	Compiled every time executed
Complexity	Can include conditional logic, loops, and error handling	Limited to simple SQL operations

Example of a Stored Procedure (SQL Server):

```
CREATE PROCEDURE GetStudentMarks
    @StudentID INT
AS
BEGIN
    SELECT Name, Marks
    FROM Students
    WHERE ID = @StudentID;
END;
```

You can execute it like this:

```
EXEC GetStudentMarks @StudentID = 1;
```

2. Explain the advantages of using stored procedures

1. Performance Improvement

- Precompiled and optimized by the database server.
- Reduces execution time for repeated tasks.

2. Reusability

- Write once, use multiple times across applications.

3. Maintainability

- Centralized logic makes updates easier.
- Changing the procedure updates all dependent applications automatically.

4. Security

- Can restrict direct access to underlying tables.
- Users can be granted permission to execute procedures without full table privileges.

5. Reduced Network Traffic

- Multiple SQL statements can be executed in one call instead of sending each query separately.

6. Error Handling and Logic

- Supports conditional statements, loops, and exception handling.
 - Helps implement complex business logic in the database.
-

In short:

Stored procedures are **predefined, reusable, and efficient SQL programs** that improve performance, security, and maintainability compared to standard SQL queries.

1. What is a view in SQL, and how is it different from a table?

Definition of a View

A view in SQL is a virtual table that is based on the result of a SQL query.

It does not store data physically (except in materialized views in some databases).

It displays data dynamically from one or more tables.

Users can query a view like a regular table.

Differences Between a View and a Table

Feature	Table	View
Storage	Physically stores data in the database	Does not store data; shows data from underlying tables
Data	Contains actual data	Contains a query that fetches data dynamically
Update	Can be updated directly (usually)	May be updatable if it meets certain conditions, otherwise read-only
Structure	Fixed structure	Defined by a SELECT query; can combine multiple tables
Purpose	Store data	Simplify complex queries, restrict access, present customized data

Example of a View:

```
CREATE VIEW TopStudents AS
SELECT Name, Marks
FROM Students
WHERE Marks > 80;
```

You can then query it like a table:

```
SELECT * FROM TopStudents;
```

2. Explain the advantages of using views in SQL databases

Simplify Complex Queries

Encapsulates complicated joins and aggregations.

Users can query the view without writing complex SQL every time.

Enhance Security

Restrict access to specific columns or rows.

Users can query the view without accessing underlying tables directly.

Data Abstraction

Provides a logical representation of data.

Users do not need to know the actual table structure.

Consistency

Ensures consistent query results for commonly used calculations or filters.

Reusability

The same view can be used by multiple applications or users.

In short:

A view is a virtual table that makes querying easier, provides security, and simplifies data presentation, whereas a table physically stores data.

1. What is a trigger in SQL? Describe its types and when they are used.

Definition of a Trigger

A **trigger** in SQL is a **special kind of stored procedure** that is **automatically executed (fired)** in response to **certain events** on a table or view.

- It is used to **enforce business rules, maintain data integrity, or automatically update related tables.**

Types of Triggers

Triggers are usually categorized based on the **event** and the **timing**:

a) Based on Timing

1. BEFORE Trigger

- Executed **before** the triggering event occurs (INSERT, UPDATE, DELETE).
- Used to **validate data** or modify values before saving.

2. AFTER Trigger

- Executed **after** the triggering event occurs.
- Used for **logging, auditing, or updating related tables.**

b) Based on Event

1. INSERT Trigger

- Fires when a new row is inserted.
- Example: Automatically assign a unique code or log insertion.

2. UPDATE Trigger

- Fires when an existing row is updated.
- Example: Track changes to salary or update modification timestamp.

3. DELETE Trigger

- Fires when a row is deleted.
- Example: Maintain history or prevent deletion of critical records.

1. What is PL/SQL, and how does it extend SQL's capabilities?

Definition of PL/SQL

- **PL/SQL (Procedural Language/SQL)** is Oracle's **procedural extension of SQL**.
- It combines the data manipulation power of SQL with **procedural programming features** like variables, loops, and conditions.
- PL/SQL allows developers to write **blocks of code** that can include SQL statements along with programming constructs.

How PL/SQL Extends SQL

Feature	SQL	PL/SQL
Programming	Declarative, non-procedural	Procedural programming (loops, IF statements)
Control Flow	No control structures	Supports IF-ELSE, FOR, WHILE loops
Variables	Cannot declare	Can declare and use variables and constants

Modularity	SQL statements only	Supports procedures, functions, triggers, and packages
Error Handling	Limited	Supports exception handling (TRY-CATCH equivalent)
Batch Processing	Executes one SQL at a time	Can execute multiple SQL statements as a single block

Example of a PL/SQL Block:

```
DECLARE
    v_total NUMBER;
BEGIN
    SELECT SUM(Salary) INTO v_total
    FROM Employees;
    DBMS_OUTPUT.PUT_LINE('Total Salary: ' || v_total);
END;
```

- Combines SQL (SELECT SUM) with procedural logic (variable declaration, output).

2. List and explain the benefits of using PL/SQL

1. Improved Productivity

- Can execute multiple SQL statements together in a **single PL/SQL block**.
- Reduces repetitive SQL coding.

2. Procedural Capabilities

- Supports **loops, conditions, and variables**, making it easier to implement complex logic.

3. **Modularity**

- Code can be organized into **procedures, functions, triggers, and packages**.
- Improves **reusability** and maintainability.

4. **Error Handling**

- Built-in **exception handling** to manage runtime errors.
- Improves reliability and prevents application crashes.

5. **Performance**

- Reduces **network traffic** by executing multiple SQL statements in one block.
- Precompiled code runs faster than executing individual SQL statements.

6. **Security**

- Sensitive logic can be encapsulated in stored procedures or packages.
- Users can be granted access to procedures without giving direct table access.

7. **Integration with SQL**

- Seamlessly integrates with SQL for data manipulation while providing procedural control.
